UNIVERSIDAD NACIONAL DE TRUJILLO

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

PROGRAMA DE ESTUDIOS DE INGENIERÍA DE SISTEMAS



TÍTULO:

PROCEDIMIENTOS ALMACENADOS Y DISPARADORES EN MYSQL Y SU INVOCACIÓN DESDE APLICACIONES JAVA-NETBEANS

AUTORES:

Aldoradin Espinoza, Alexia Anaís

Enco Tirado, Xiomara Nayelly

Sánchez Mego, Joaquín André

Romero Pompa, Flor Jazmín

Verastegui Lezama, Harold Joseph

ASESORA:

Ms. Ing. Vidal Melgarejo, Zoraida Yanet

Experiencia Curricular: Programación Orientada a Objetos II

Ciclo Académico:

III Ciclo

Fecha de entrega: 5 de Junio de 2026

Guadalupe - Perú
2026

DEDICATORIA

A Dios, por brindarnos la vida, la salud y la fortaleza espiritual necesaria para superar cada uno de los retos académicos y personales que se presentan en nuestro camino.

A nuestros padres y familiares, quienes con su amor incondicional, esfuerzo constante y sabios consejos constituyen el pilar fundamental de nuestra educación y nos motivan diariamente a buscar la excelencia profesional.

A nuestra asesora, la Ms. Ing. Zoraida Yanet Vidal Melgarejo, por compartir con nosotros su valioso conocimiento técnico, su paciencia y su guía metodológica durante el desarrollo de la experiencia curricular de Programación Orientada a Objetos II.

ÍNDICE

DEDICATORIA II
RESUMEN / ABSTRACT IV

PROCEDIMIENTOS ALMACENADOS Y DISPARADORES EN MYSQL Y SU INVOCACIÓN DESDE APLICACIONES JAVA-NETBEANS 1

1.1. Planteamiento del problema 1

1.2. Justificación Técnica 1

1.3. Objetivos de la investigación 1

1.4. Alcance y Limitaciones 2

1.5. Organización del Documental 2

CAPÍTULO II: MARCO TEÓRICO Y CONCEPTUAL..... 3

2.1. Antecedentes de la Investigación 3

2.2. Conceptos Claves 3

2.2.1. Bases de datos y MySQL 3

2.2.2. Procedimientos Almacenados (Stored Procedures) 3

2.2.3. Disparadores (Triggers).....4

2.3. Diferencias operativas 5

2.4. Modelos y Metodologías del Proceso de Desarrollo (SDLC) 5

2.5. Herramientas y estándares aplicables (IEEE, ISO/IEC) 5

2.6. Relación con la Ingeniería de Sistemas 6

CAPÍTULO III: METODOLOGÍA 7

3.1. Enfoque y Diseño de la investigación 7

3.2. Criterios de Selección de Fuentes y Literatura 7

3.3. Herramientas Tecnológicas y Entorno de Pruebas 7

3.4. Fases del procedimiento Metodológico 7

CAPÍTULO IV: DESARROLLO Y ANÁLISIS 8

4.1. Implementación de la Base de Datos en MySQL 8

4.2. Creación del Procedimiento Almacenado y Disparador ……..... 9

4.3. Implementación y Conexión en Java mediante Apache NetBeans ……..10

4.4. Verificación del Sistema ante Diferentes Situaciones de Venta ……14

4.5. Síntesis Integradora del Capítulo ……14

CAPÍTULO V: RESULTADOS 11

5.1. Evaluación de la Ejecución de la API 11

5.2. Verificación de la Integridad Automática en el Servidor 11

CAPÍTULO VI: DISCUSIÓN 1

6.1. Análisis de Seguridad e Inyección SQL 11

6.2. Verificación del Acoplamiento y Portabilidad de software 11

6.3. Áreas de Investigación Futura 11

CAPÍTULO VII: CONCLUSIONES Y RECOMENDACIONES 12

7.1. Conclusión 12

7.2. Recomendaciones.....12

REFERENCIAS 13
APÉNDICES 14

Apéndice A: Arquitectura Conceptual y Evidencias Gráficas 14

III

RESUMEN

El presente informe de investigación aborda un problema muy común en el desarrollo de software actual: cuando creamos aplicaciones y dejamos que toda la lógica del negocio o el procesamiento de datos se maneje solo en el lado del cliente (la aplicación en Java), el sistema se vuelve lento por el constante tráfico de red, el código se duplica y la base de datos queda expuesta a fallos de seguridad. Para solucionar esto, como estudiantes de Ingeniería de Sistemas de la UNT, decidimos evaluar de forma práctica cómo los Procedimientos Almacenados y los Triggers en MySQL mejoran la arquitectura de un sistema.

Para demostrarlo, desarrollamos un caso práctico centrado en un sistema de control de ventas e inventario conectado mediante Apache NetBeans. Los resultados muestran que, al programar estas rutinas directamente en el servidor de la base de datos, el rendimiento mejora significativamente, ya que se reduce la transmisión de datos a través de la red y se bloquean amenazas como la inyección SQL. Además, confirmamos que conectar estas herramientas mediante la interfaz JDBC CallableStatement es fundamental para garantizar la seguridad y la eficiencia de la aplicación. Como futuros ingenieros, concluimos que dominar estas técnicas de persistencia es vital para desarrollar software real, seguro y escalable.

Palabras clave: *Procedimientos almacenados, disparadores, MySQL, CallableStatement, JDBC.*

ABSTRACT

This research report addresses a very common problem in modern software development: when we create applications and leave all the business logic or data processing to be handled solely on the client side (the Java application), the system becomes slow due to constant network traffic, code is duplicated, and the database is exposed to security vulnerabilities. To solve this, as Systems Engineering students at UNT, we decided to practically evaluate how Stored Procedures and Triggers in MySQL improve a system's architecture.

To demonstrate this, we developed a case study focused on a sales and inventory control system connected via Apache NetBeans. The results show that by programming these routines directly on the database server, performance improves significantly because data travel across the network is reduced, and threats such as SQL injection are blocked. Furthermore, we confirmed that connecting these tools through the JDBC CallableStatement interface is key to ensuring the application's security and efficiency. As future engineers, we concluded that mastering these

persistence techniques is vital for building real, secure, and scalable software.

Keywords: *Stored procedures, triggers, MySQL, CallableStatement, JDBC.*

IV

PROCEDIMIENTOS ALMACENADOS Y DISPARADORES EN MYSQL Y SU INVOCACIÓN DESDE APLICACIONES JAVA-NETBEANS

En el contexto de la carrera de Ingeniería de Sistemas, uno de los aspectos más importantes dentro del desarrollo de software empresarial es la correcta gestión y persistencia de la información. En los sistemas actuales, el funcionamiento depende en gran medida de cómo se almacenan, consultan y administran los datos durante su ciclo de vida. De forma tradicional, en las arquitecturas cliente-servidor, la mayor parte de la lógica de procesamiento se ejecuta en el lado del cliente, mientras que la base de datos cumple principalmente el rol de almacenamiento. No obstante, el aumento progresivo del volumen de transacciones en aplicaciones modernas ha puesto en evidencia ciertas limitaciones de este enfoque, sobre todo en el rendimiento y la escalabilidad del sistema.

Frente a esta problemática, cobran importancia las funcionalidades de programación del lado del servidor en MySQL, ya que permiten trasladar parte de la lógica de control y validación directamente a la base de datos, haciendo que esta participe de manera más activa en los procesos del sistema. En este sentido, el desarrollo de software actual requiere arquitecturas más eficientes, donde la persistencia de datos no solo sea estable, sino también consistente y optimizada, especialmente en entornos donde se manejan múltiples transacciones de forma simultánea.

En muchos casos, cuando toda la lógica de validación, reglas de negocio y cálculos se dejan únicamente en la aplicación cliente (por ejemplo en Java), empiezan a aparecer problemas bastante comunes en entornos reales: consultas repetitivas, mayor latencia en la comunicación con el servidor y ciertos riesgos de seguridad por exposición de lógica en el lado del cliente. Este problema se vuelve más evidente cuando se manejan operaciones relacionadas entre sí, como registrar una venta, actualizar el inventario y generar comprobantes, ya que al ejecutarlas como sentencias separadas desde la aplicación se pierde eficiencia, consistencia y control transaccional. Por ello, la presente investigación se justifica en la necesidad de trasladar parte de esa lógica directamente al servidor de base de datos, con el fin de mejorar el rendimiento general del sistema, asegurar la integridad de los datos y reducir la carga en la comunicación entre cliente y servidor.

Por consiguiente, el objetivo general de este estudio es analizar el impacto en la arquitectura, el rendimiento y la seguridad al trasladar la lógica de negocio hacia el servidor de base de datos, mediante el uso de procedimientos almacenados en MySQL e integrándose posteriormente con aplicaciones desarrolladas en Java. En este caso, el desarrollo se realiza utilizando el entorno de Apache NetBeans para la

programación en Java y MySQL Workbench para la gestión y ejecución de las consultas en la base de datos. La idea es evaluar cómo este enfoque puede mejorar la organización del sistema en comparación con una lógica completamente manejada desde la aplicación. Para lograrlo, se plantean los siguientes objetivos específicos:

1. Analizar los conceptos básicos y las diferencias operativas entre procedimientos almacenados y disparadores, con la finalidad de determinar su correcta aplicación dentro del diseño del sistema.
2. Describir el proceso de creación, estructura y ejecución de rutinas almacenadas en el motor transaccional InnoDB de MySQL, considerando su uso en entornos reales.
3. Implementar la ejecución de procedimientos almacenados desde Java utilizando la clase CallableStatement de JDBC, asegurando el paso correcto de parámetros y la interacción con la base de datos.

El alcance de esta investigación se centra en el diseño de una base de datos relacional y la programación del lado del servidor aplicada a un sistema de ventas e inventario, trabajando en un entorno local de pruebas con MySQL InnoDB y el JDK 27 de Java. En este caso, el enfoque está orientado principalmente a entornos controlados donde se pueda evaluar el comportamiento de las rutinas almacenadas sin interferencias externas. Como limitación, no se consideran arquitecturas distribuidas, sistemas en la nube ni el uso de bases de datos NoSQL, ya que el objetivo es mantener el análisis dentro del modelo relacional tradicional.

Finalmente, el documento se organiza en capítulos. El Capítulo I corresponde a la introducción; el Capítulo II desarrolla el marco teórico, donde se abordan los conceptos y fundamentos principales relacionados con el tema; el Capítulo III presenta la metodología utilizada para el desarrollo de la investigación; y el Capítulo IV describe el proceso de implementación del sistema paso a paso. Posteriormente, los Capítulos V, VI y VII presentan los resultados obtenidos, el análisis correspondiente y las conclusiones finales del estudio, respectivamente, seguidos de las referencias bibliográficas y los anexos.

Finalmente, el documento se organiza en capítulos:

- El **Capítulo I** corresponde a la introducción.
- El **Capítulo II** desarrolla el marco teórico, donde se abordan los conceptos y fundamentos principales relacionados con el tema.
- El **Capítulo III** presenta la metodología utilizada para el desarrollo de la investigación.
- El **Capítulo IV** describe el proceso de implementación del sistema paso a paso.
- Los **Capítulos V, VI y VII** presentan los resultados obtenidos, el análisis correspondiente y las conclusiones finales del estudio, respectivamente, seguidos de las referencias bibliográficas y los anexos.

CAPÍTULO II: MARCO TEÓRICO

2.1. Antecedentes de la investigación

La necesidad de optimizar el acceso a datos ha sido objeto de estudio continuo. A nivel nacional e internacional, investigaciones como la de Quintero et al. (2008) sobre directrices para la construcción de artefactos de persistencia demuestran que centralizar la validación de los datos en el servidor reduce errores lógicos y mejora la concurrencia. De manera más reciente, estudios como el de López y García (2023) sobre optimización de consultas SQL en sistemas distribuidos reafirman que el uso de rutinas almacenadas reduce los tiempos de respuesta del servidor hasta en un 40% al minimizar los viajes de red (round-trips) entre el aplicativo y el motor de base de datos.

2.2. Conceptos Claves

2.2.1. Bases de datos y MySQL

Una base de datos relacional es un sistema de almacenamiento de información basado en el modelo propuesto por Edgar F. Codd, el cual organiza los datos en tablas compuestas por filas y columnas. Estas tablas se relacionan entre sí mediante claves primarias y foráneas, lo que permite mantener la integridad de la información y reducir la redundancia.

MySQL es un sistema de gestión de bases de datos relacional (RDBMS) de código abierto que funciona bajo una arquitectura cliente-servidor. Este sistema utiliza el lenguaje SQL (*Structured Query Language*) para realizar operaciones de consulta y manipulación de datos, mientras que MySQL corresponde al motor que interpreta y

ejecuta dichas instrucciones dentro del sistema de base de datos (Oracle, 2024).

En el desarrollo de software, MySQL destaca por su soporte para transacciones ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que garantiza que las operaciones sobre los datos se ejecuten de forma completa y segura. Esto resulta especialmente importante en sistemas transaccionales, donde se requiere mantener la integridad de la información ante múltiples operaciones simultáneas o posibles fallos del sistema (Oracle, 2024).

2.2.2. Procedimientos Almacenados (Stored Procedures)

Un procedimiento almacenado es un conjunto de instrucciones SQL precompiladas y almacenadas físicamente en el diccionario de datos del servidor (Oracle, 2026). Como características principales, se trata de bloques de código reutilizables que permiten organizar y encapsular la lógica de las consultas dentro del propio gestor de base de datos. Para interactuar dinámicamente con la aplicación cliente, estas rutinas admiten el uso de parámetros de entrada (IN), parámetros de salida (OUT) y variables bidireccionales (INOUT). A diferencia de otras operaciones, su ejecución no es autónoma, sino que requieren ser invocados explícitamente por el usuario o el aplicativo mediante la sentencia SQL CALL.

En cuanto a sus ventajas, estos procedimientos mejoran significativamente el rendimiento general del sistema al reducir el tráfico de red, ya que la comunicación se limita a la invocación de la rutina en lugar de múltiples sentencias SQL enviadas desde la aplicación cliente. Adicionalmente, proporcionan una capa de seguridad más robusta al ocultar la estructura interna de las tablas y restringir el acceso directo a los datos subyacentes. En este sentido, el motor de base de datos implementa mecanismos de control de acceso específicos para estas rutinas, permitiendo asignar

privilegios detallados sobre su ejecución o modificación.

Por otro lado, su principal desventaja radica en que la lógica de negocio implementada directamente en el servidor genera un alto nivel de acoplamiento con el sistema gestor de base de datos utilizado. Asimismo, debido a que MySQL incorpora extensiones propias y ciertas diferencias respecto al estándar SQL, el código puede volverse difícil de migrar hacia otros motores relacionales sin requerir una reestructuración significativa.

2.2.3. Disparadores (Triggers)

Los disparadores son rutinas almacenadas que no se invocan manualmente, sino que reaccionan de manera autónoma ante eventos de modificación de datos o DML (Oracle, 2026). Estructuralmente, se asocian a una tabla específica y pueden activarse en momentos definidos como BEFORE o AFTER ante operaciones como INSERT, UPDATE o DELETE. Estas rutinas operan a nivel de fila (FOR EACH ROW), lo que implica que se ejecutan de manera individual por cada registro afectado por la operación que las desencadena. Además, en MySQL utilizan los modificadores NEW y OLD, los cuales permiten acceder a los valores antes y después de la modificación de los datos.

Entre sus beneficios más relevantes, destacan por ser un mecanismo adecuado para garantizar reglas de negocio de forma automática, asegurando la integridad de los datos de manera consistente sin depender de la aplicación cliente. De igual forma, facilitan la automatización de procesos en segundo plano, como auditorías o registro de cambios, lo cual resulta útil en sistemas donde se requiere trazabilidad de la información. Como consecuencia adicional, reducen la carga en la aplicación cliente al evitar la necesidad de ejecutar instrucciones complementarias tras cada operación.

Por otro lado, entre sus desventajas, su naturaleza implícita dificulta la depuración, ya que sus acciones no se reflejan directamente en el flujo de ejecución de la aplicación. Finalmente, un diseño inadecuado puede generar efectos en cascada o bucles de ejecución no deseados, afectando el rendimiento general del sistema de base de datos.

2.3. Diferencias operativas

Para un correcto diseño arquitectónico, el ingeniero debe diferenciar su aplicación:

Tabla 1

Diferencias operativas y estructurales entre Procedimientos y Disparadores

Característica	**Procedimiento Almacenado**	**Disparador (Trigger)**
Ejecución	Manual o invocada explícitamente desde la API.	Totalmente automática en respuesta a un evento.
Invocación	A través de la sentencia SQL CALL.	Activación por operaciones DML de la tabla.
Control	Control total y directo por el programador.	Control administrado directamente por el

		RDBMS.
Parámetros	Admiten el uso de parámetros externos (IN, OUT).	No admiten parámetros, dependen del contexto (NEW/OLD).

Nota. Elaboración propia basada en la documentación oficial de referencia técnica (Oracle, 2026).

2.4. Modelos y Metodologías del Proceso de Desarrollo (SDLC)

La implementación de la persistencia de datos y la programación del lado del servidor deben alinearse con el Ciclo de Vida de Desarrollo de Software (SDLC). En los modelos tradicionales o predictivos (como el Modelo en Cascada), el diseño del esquema relacional y las rutinas almacenadas se consolidan de forma estricta durante las fases iniciales de diseño arquitectónico, previo a la codificación del cliente.

Por el contrario, en las metodologías ágiles (como Scrum o Kanban), la base de datos evoluciona incrementalmente. En este contexto, los procedimientos almacenados y disparadores facilitan la refactorización del sistema, ya que cualquier modificación en las reglas de negocio complejas se realiza directamente en el servidor MySQL. Esto evita la necesidad de recompilar y redistribuir continuamente la aplicación cliente en Java, adaptándose de forma fluida a las entregas continuas de software funcional.

2.5. Herramientas y estándares aplicables (IEEE, ISO/IEC) Para la implementación y gestión de la base de datos, así como para el desarrollo del software transaccional, se emplea un conjunto de herramientas alineadas con estándares utilizados en la industria del desarrollo de sistemas. En la capa de persistencia se utiliza MySQL Workbench 8.0.47 (edición comunitaria para Microsoft Windows de 64 bits), que

corresponde a la interfaz gráfica oficial para el diseño de esquemas, administración de objetos relacionales y creación de rutinas en el servidor.

Por otro lado, el desarrollo de la aplicación cliente se realiza mediante el entorno de desarrollo integrado Apache NetBeans IDE 27, ejecutado sobre el JDK 25 (Java Development Kit). Este entorno facilita la organización del proyecto bajo una arquitectura en capas, separando la lógica de presentación, la lógica de negocio y el acceso a datos, lo que contribuye a una mejor estructura, mantenibilidad y escalabilidad del sistema.

Desde el punto de vista normativo, la sintaxis y funcionamiento de las rutinas almacenadas en el servidor, así como la interacción desde Java mediante la interfaz CallableStatement, se basan en el estándar ISO/IEC 9075 (Database Language SQL), el cual establece las reglas generales del lenguaje SQL a nivel internacional. Asimismo, el diseño del sistema también se alinea con el estándar ISO/IEC 25010 (System and Software Quality Models), especialmente en lo relacionado con la calidad del software.

En este contexto, el uso de procedimientos almacenados en el servidor contribuye a mejorar la eficiencia del sistema, reduciendo la latencia en la comunicación cliente-servidor. Además, refuerza la seguridad del sistema al disminuir la exposición directa de consultas SQL en la aplicación, ayudando a mitigar riesgos como la inyección SQL mediante el uso de sentencias precompiladas y controladas desde la base de datos.

2.6. Relación con la Ingeniería de Sistemas

La orquestación de estas herramientas a nivel de servidor junto con el lenguaje Java es clave para el diseño integral de soluciones tecnológicas. Desde una perspectiva

arquitectónica, el encapsulamiento del código promueve la construcción de sistemas altamente modulares. Al respecto, Deitel y Deitel (2012) afirman que "el uso de interfaces y procedimientos bien definidos aísla a los programas de aplicación de los detalles de la base de datos subyacente" (p. 1042).

A su vez, esta separación de capas responde a las necesidades críticas del desarrollo empresarial contemporáneo. Las bases de datos relacionales se conciben como el pilar fundamental que permite a las organizaciones automatizar sus procesos más importantes (Fernández y Giraldo, 2011). Por ende, centralizar la lógica y las reglas de negocio en el gestor a través de procedimientos y disparadores garantiza resultados fiables, dado que la seguridad e integridad de los datos son esenciales para el buen funcionamiento y la toma de decisiones estratégicas dentro de cualquier organización (Fernández y Giraldo, 2011). Bajo este enfoque, aprovechar al máximo la potencia del gestor asegura una manipulación efectiva de la información corporativa (Fernández y Giraldo, 2011).

2.7. Fases de Creación en MySQL e Invocación desde Java

Para poder aprovechar los beneficios de la programación del lado del servidor, es necesario entender cómo se da todo el flujo de trabajo en la práctica. En este sentido, la implementación de la lógica transaccional y su consumo desde una aplicación cliente se realiza mediante dos entornos principales: el sistema gestor de base de datos y el entorno de desarrollo Java, utilizando la API JDBC (Java Database Connectivity).

2.7.1. Estructuración en MySQL La creación de un procedimiento almacenado en MySQL requiere seguir una sintaxis específica y bien definida dentro del motor de base de datos. En la práctica, este proceso se desarrolla mediante una serie de pasos

ordenados que aseguran su correcta compilación y ejecución (Oracle, 2026):

- **Redefinición del delimitador:** En primer lugar, se modifica temporalmente el delimitador estándar (por ejemplo $$ o //), con el fin de permitir que el servidor intérprete todo el bloque de código sin cortar la ejecución antes de tiempo.
- **Declaración y parámetros:** Luego se define la rutina mediante CREATE PROCEDURE o CREATE TRIGGER, asignándole un nombre y especificando los parámetros necesarios, como por ejemplo los de entrada (IN).
- **Encapsulamiento del cuerpo lógico:** A continuación, toda la lógica del procedimiento se organiza dentro de un bloque BEGIN ... END, donde se colocan las sentencias SQL que representan la lógica de negocio o transaccional.
- **Restauración del entorno:** Finalmente, se vuelve a establecer el delimitador original con DELIMITER ;, para que el servidor continúe ejecutando consultas de manera normal.

2.7.2. Pasos para la invocación desde Java con JDBC Una vez que el procedimiento ya se encuentra creado y almacenado en el servidor, su ejecución desde Java se realiza mediante JDBC, que es el conjunto de clases e interfaces del paquete java.sql. Este proceso sigue una secuencia lógica bien definida para garantizar la correcta comunicación con la base de datos (Deitel y Deitel, 2012):

- **Establecimiento de la conexión:** Primero se utiliza la clase DriverManager, que se encarga de ubicar el driver correspondiente (mysql-connector-j) y establecer la conexión con el servidor a partir de la URL, usuario y contraseña. Como resultado, se obtiene un objeto Connection.
- **Preparación de la llamada:** Para ejecutar rutinas almacenadas con parámetros, se utiliza la interfaz CallableStatement. Esta trabaja con la sintaxis

{call nombre_rutina(?, ?)}, donde los signos de interrogación funcionan como marcadores de posición (Oracle, 2023).

- **Asignación de parámetros posicionales:** Antes de ejecutar la llamada, cada parámetro se asigna mediante métodos específicos según su tipo de dato, como setInt() o setString(). Este paso es clave porque ayuda a prevenir inyecciones SQL al separar la lógica del código de los datos (Baeldung, 2023).
- **Ejecución y cierre de recursos:** Finalmente, se ejecuta la instrucción mediante execute(), enviando los parámetros al servidor de forma segura. Además, se recomienda manejar la excepción SQLException y cerrar la conexión una vez terminada la operación para evitar bloqueos en el sistema (Horstmann, 2022).

CAPÍTULO III

METODOLOGÍA

3.1. Enfoque y diseño de la investigación

La presente investigación se enmarca bajo un enfoque cualitativo con nivel aplicativo, orientado a la resolución de un problema técnico relacionado con la arquitectura de software en sistemas de gestión de bases de datos. Este enfoque es adecuado debido a que el estudio se centra en el análisis del comportamiento de la interacción entre el sistema gestor MySQL y la aplicación cliente desarrollada en Java, sin recurrir a mediciones estadísticas complejas, sino a la observación del funcionamiento del sistema en un entorno controlado.

El diseño es de tipo descriptivo y experimental, lo que permite la replicabilidad del estudio mediante la documentación detallada de la infraestructura tecnológica y las fases del procedimiento implementado. Es descriptivo porque se especifican las

características del sistema, las herramientas utilizadas y la estructura de la base de datos; y es experimental porque se realizan pruebas controladas sobre la ejecución de procedimientos almacenados y disparadores, evaluando su comportamiento en tiempo real dentro del entorno MySQL.

3.2. Criterios de selección de fuentes y herramientas

La recopilación de la información y la definición del entorno de trabajo se realizaron mediante un proceso sistemático y fundamentado:

- **Criterios de selección bibliográfica:** El proceso inició con la revisión de un conjunto de referencias base proporcionadas en el curso, seleccionando aquellos libros y artículos directamente alineados con la temática transaccional asignada. Para fortalecer el sustento teórico, se amplió la búsqueda en repositorios académicos confiables como SciELO, Redalyc, Dialnet y Google Académico , priorizando fuentes relevantes tanto a nivel internacional como latinoamericano. Asimismo, debido al carácter práctico del trabajo, se complementó con documentación técnica oficial. En este sentido, se consultó el manual de Oracle para validar la sintaxis del servidor, Baeldung como referencia en buenas prácticas en Java y MySQL Tutorial para comprender la estructuración de rutinas y consultas más complejas.
- **Criterios de inclusión y lenguaje:** Se priorizó el uso de fuentes en idioma español para asegurar una mejor comprensión de los conceptos teóricos y evitar posibles errores de interpretación. No obstante, para aspectos técnicos específicos como la sintaxis de programación, el manejo de excepciones y el uso de APIs (como JDBC), se recurrió a documentación en inglés, ya que esta suele estar más actualizada y alineada con los estándares reales del desarrollo de software.

- **Selección del entorno y herramientas:** La elección del entorno tecnológico se basó en criterios de rendimiento, estabilidad y aplicabilidad en contextos reales. En la capa de base de datos, se utilizó MySQL Server con el motor de almacenamiento InnoDB, debido a su soporte para transacciones bajo el modelo ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), así como su capacidad de bloqueo a nivel de fila, lo cual resulta clave para mantener la integridad de los datos durante la ejecución de procedimientos y disparadores. La administración se realizó mediante MySQL Workbench 8.0.47. Por otro lado, el desarrollo del aplicativo cliente se llevó a cabo en Apache NetBeans IDE 27, utilizando el JDK 25, lo que permitió organizar el proyecto de manera estructurada y facilitar la integración con la base de datos a través de la interfaz CallableStatement de JDBC.

3.3. Herramientas tecnológicas

Se seleccionó MySQL Server como sistema gestor de base de datos, utilizando el motor InnoDB por su soporte nativo para transacciones bajo el modelo ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo cual garantiza la integridad de los datos durante la ejecución de operaciones críticas.

Para el desarrollo del lado del cliente se utilizó Apache NetBeans IDE 27 sobre JDK 25, lo que permitió organizar el proyecto de forma estructurada. La conexión entre ambas capas se realizó mediante el controlador JDBC (mysql-connector-j), asegurando una comunicación directa y desacoplada entre Java y el servidor de base de datos.

3.4. Procedimiento

El desarrollo del proyecto y el análisis de la conexión se realizaron a través de cinco

fases ordenadas, pensadas para asegurar el buen funcionamiento, la seguridad y la consistencia de los datos en el sistema:

Fase 1: Modelado físico de la base de datos: Consistió en crear la base de datos tienda_db en el servidor MySQL. Definimos las tablas principales Productos (maestro) y Ventas (transacciones), aplicando las claves primarias, claves foráneas y la restricción CHECK para que el motor InnoDB controle que el stock no sea negativo.

Fase 2: Creación de las rutinas en el servidor: Programamos y guardamos el código directamente en el servidor de la base de datos. Aquí creamos el procedimiento almacenado sp_registrar_venta para registrar las ventas de forma estándar y el disparador trg_actualizar_stock para que el inventario se descuente automáticamente en segundo plano.

Fase 3: Configuración de la conectividad en Java: Configuramos la comunicación entre la aplicación y la base de datos. Para lograrlo, agregamos el conector mysql-connector-j en el entorno de Apache NetBeans, colocando la dirección del servidor, el usuario y la contraseña para habilitar la conexión local.

Fase 4: Desarrollo del código en NetBeans y control de errores: Escribimos el código en Java utilizando la interfaz CallableStatement para pasar los datos de forma segura (lo que evita ataques de inyección SQL). Además, usamos la estructura *try-with-resources* para asegurar que las conexiones se cierren solas y no dejen el sistema lento.

Fase 5: Pruebas y validación del sistema: Corrimos el programa desde Java para comprobar el flujo completo. Luego, hicimos consultas SELECT directas en MySQL Workbench antes y después de ejecutar el código, verificando el comportamiento del

sistema en dos situaciones: cuando hay stock y cuando el stock es insuficiente.

CAPÍTULO IV

DESARROLLO Y ANÁLISIS

4.1. Implementación de la Base de Datos en MySQL

En el desarrollo de software actual, es muy importante decidir dónde se va a procesar la información. Tradicionalmente, la aplicación en Java se encargaba de todo el trabajo y enviaba muchas consultas sueltas, lo que ponía lento el sistema. Para solucionar esto, proponemos un diseño donde la base de datos participe activamente en el control de la información mediante programación del lado del servidor.

4.1.1. Análisis Comparativo: Lógica en el Cliente vs. Lógica en el Servidor

Al analizar cómo manejar los datos del sistema, comparamos las ventajas y desventajas de cada enfoque:

- **Manejar todo en el cliente (Java):** Aunque hace que el código sea más independiente del motor que usemos, obliga a la aplicación a enviar consultas a cada rato a través de la red. Esto aumenta la demora en responder y expone cómo están armadas nuestras tablas dentro del código.
- **Manejar la lógica en el servidor (MySQL):** Al guardar los procedimientos y disparadores en la base de datos, la aplicación solo tiene que mandar a llamar la rutina. Esto reduce drásticamente el tráfico en la red y garantiza que las reglas de negocio se cumplan siempre, sin importar qué aplicación intente modificar los datos.

4.1.2. Caso de Estudio: Sistema de Control de Ventas e Inventario (*tienda_db*)

Para demostrar este funcionamiento de forma práctica, trabajamos con un modelo de dos tablas conectadas que dependen una de la otra:

1. **Tabla Productos:** Guarda la lista de productos y la cantidad disponible en el inventario.
2. **Tabla Ventas:** Registra cada una de las salidas de productos del sistema.

Este es el script completo que ejecutamos en MySQL para crear la estructura:

```
CREATE DATABASE tienda_db;

USE tienda_db;



CREATE TABLE Productos (

    id_producto INT PRIMARY KEY,

    nombre_producto VARCHAR(50) NOT NULL,

    stock INT NOT NULL CHECK (stock >= 0) -- Evita que el stock baje de cero

);



CREATE TABLE Ventas (

    id_venta INT AUTO_INCREMENT PRIMARY KEY,

    id_producto INT,
```

```
    cantidad_vendida INT NOT NULL,
    fecha_venta DATETIME NOT NULL,
    FOREIGN KEY (id_producto) REFERENCES Productos(id_producto)
    ON DELETE RESTRICT ON UPDATE CASCADE -- Mantiene la relación de los datos
);

INSERT INTO Productos VALUES (101, 'Teclado Mecánico', 50);
```

Síntesis integradora de la sección: Crear las tablas con sus relaciones y restricciones (como el CHECK) es el primer paso para proteger la información. Sin embargo, para automatizar los procesos y asegurar que registrar una venta y descontar el stock ocurran como una sola acción, necesitamos programar funciones dentro de la misma base de datos, como veremos a continuación.

4.2. Creación del Procedimiento Almacenado y Disparador

Para que nuestro sistema funcione correctamente, necesitamos que cada venta descuente el stock de inmediato. Si dejamos que el programador de Java recuerde hacer el descuento manualmente, corremos el riesgo de que ocurran errores. Por eso, creamos una solución dividida en dos partes: una llamada manual y una respuesta automática.

4.2.1. Registro de Ventas mediante el Procedimiento Almacenado

Implementamos el procedimiento sp_registrar_venta. Su única tarea es recibir el

código del producto y la cantidad, y guardarlos en la tabla Ventas. Al usar parámetros de entrada (IN), el sistema se encarga de poner la fecha actual con NOW(), haciendo que el código en Java sea más sencillo.

4.2.2. Actualización de Stock mediante el Disparador (Trigger)

Para que el stock se actualice solo, programamos el disparador trg_actualizar_stock. Este código se ejecuta automáticamente en segundo plano justo después (AFTER INSERT) de que se guarde una nueva venta en el sistema.

Este es el código que utilizamos en el servidor para crear ambas rutinas:

```
DELIMITER $$



CREATE PROCEDURE sp_registrar_venta (
    IN p_id_prod INT,
    IN p_cant INT
)
BEGIN
    INSERT INTO Ventas(id_producto, cantidad_vendida, fecha_venta)
    VALUES (p_id_prod, p_cant, NOW());
END$$
```

```
DELIMITER ;



DELIMITER $$



CREATE TRIGGER trg_actualizar_stock

AFTER INSERT ON Ventas

FOR EACH ROW

BEGIN



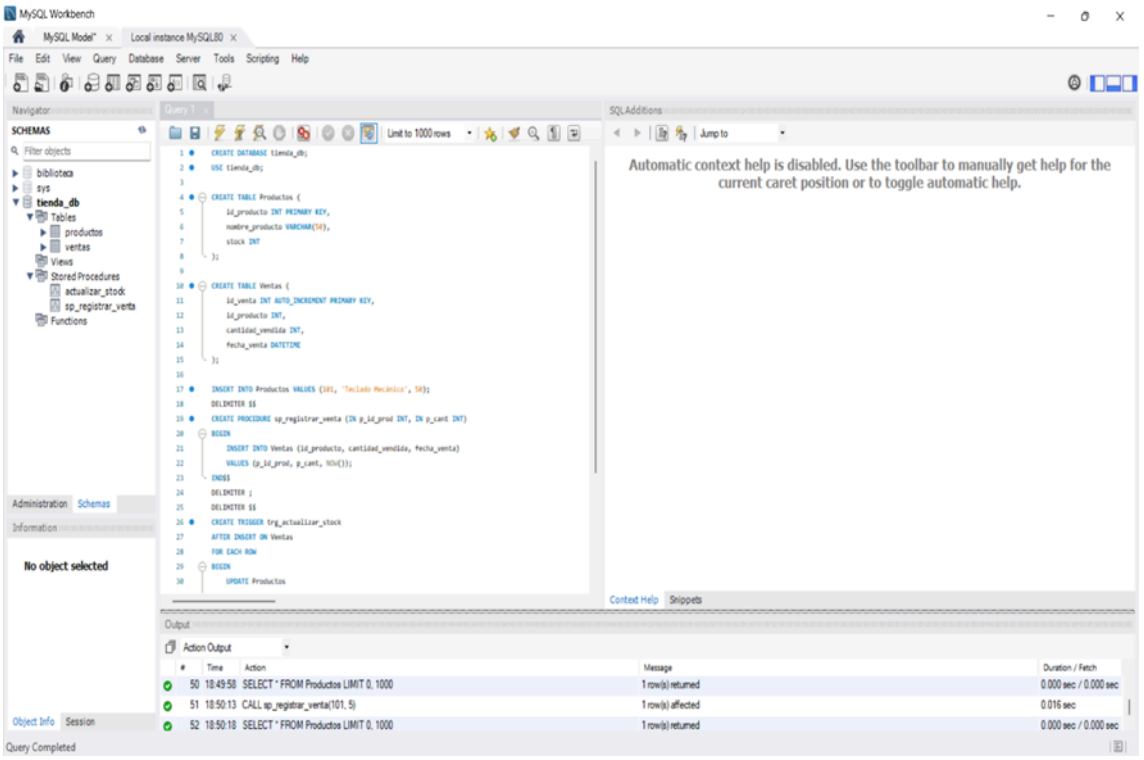
    UPDATE Productos

    SET stock = stock - NEW.cantidad_vendida

    WHERE id_producto = NEW.id_producto;

END$$

DELIMITER ;
```



4.2.3. Explicación del funcionamiento de NEW en las transacciones

El uso de la palabra NEW dentro del disparador es clave. NEW representa la fila de la venta que se acaba de registrar. Gracias a esto, el disparador sabe exactamente qué producto se vendió y qué cantidad restar. Como el disparador corre dentro de la misma operación del INSERT, si la actualización del stock falla por alguna razón, toda la venta se cancela automáticamente, evitando que la base de datos quede con datos descuadrados.

Síntesis integradora de la sección: Juntar el procedimiento y el disparador en el servidor hace que el sistema sea independiente del lenguaje de programación.
La aplicación en Java ya no necesita saber cómo se descuenta el stock ni qué tablas se modifican; solo debe llamar a la rutina de forma segura.

4.3. Implementación y Conexión en Java mediante Apache NetBeans

Con los códigos listos en MySQL, la conexión desde Java se realiza usando la librería JDBC, conectando ambas partes a través del controlador mysql-connector-j.

4.3.1. Seguridad en la Conexión y Uso de CallableStatement

En lugar de enviar texto plano desde Java, usamos la interfaz especial CallableStatement. Esto lo hacemos por dos razones importantes de seguridad:

- Está preparada exclusivamente para ejecutar comandos de tipo {call nombre_procedimiento}.
- Utiliza signos de interrogación (?) como marcadores de posición. Esto obliga a Java a revisar y limpiar los datos antes de mandarlos, lo que bloquea por completo los ataques de inyección SQL, ya que ningún texto malicioso podrá ejecutarse como comando.

4.3.2. Código Fuente de la Aplicación en Java

El programa está estructurado con la instrucción *try-with-resources* (los paréntesis en el try), lo que asegura que las conexiones se cierren de forma automática al terminar y no consuman memoria innecesaria:

```
package sistema;

import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.CallableStatement;

import java.sql.SQLException;
```

```
public class GestionVentas {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/tienda_db";

        String user = "root";

        String password = ""; // Contraseña de su servidor local

        try (Connection con = DriverManager.getConnection(url, user, password)) {

            System.out.println("Conexión a la base de datos establecida.");

            String query = "{call sp_registrar_venta(?, ?)}";



            try (CallableStatement cs = con.prepareCall(query)) {

                cs.setInt(1, 101); // Código del producto: Teclado Mecánico

                cs.setInt(2, 5);   // Cantidad a comprar

                cs.execute();

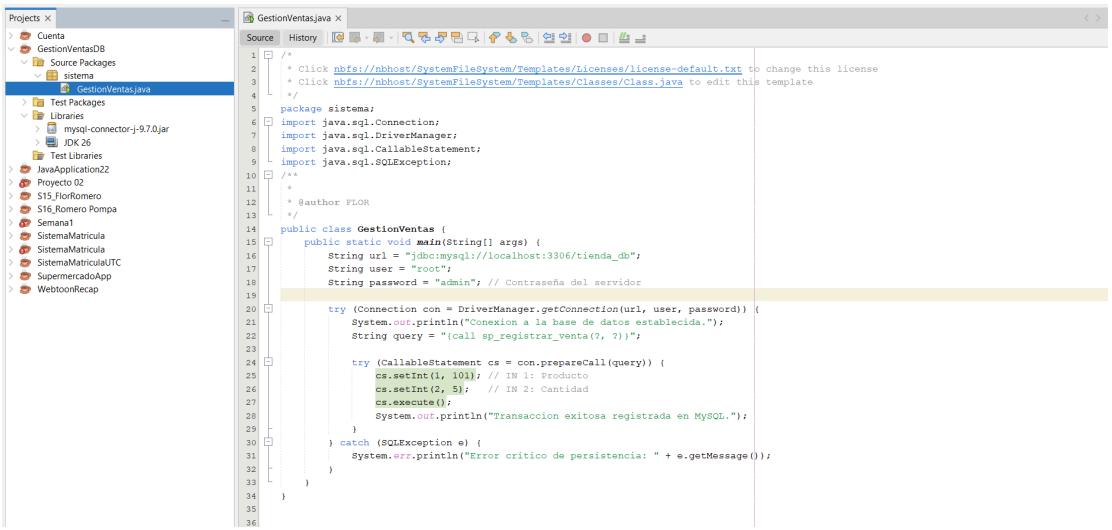
                System.out.println("Transacción exitosa registrada en MySQL.");

            }



        } catch (SQLException e) {
```

```
            System.err.println("Error crítico de persistencia: " + e.getMessage());
        }
    }
}
```



4.3.3. Sincronización del Flujo de Datos

Cuando en Java se ejecuta el método cs.execute(), la aplicación espera un momento y le pasa el control a MySQL. El servidor ejecuta el procedimiento, el disparador descuenta el stock y, si todo sale bien, MySQL le avisa a Java que el proceso terminó con éxito. En ese instante, la aplicación en Java continúa su camino normalmente.

Síntesis integradora de la sección: Conectar CallableStatement con un manejo controlado de errores es la mejor forma de integrar sistemas. Java se encarga de mostrar los mensajes y controlar la pantalla, mientras que MySQL se encarga de procesar y mantener seguros los datos guardados.

4.4. Verificación del Sistema ante Diferentes Situaciones de Venta

Para comprobar que nuestro sistema realmente responde bien en la práctica, pusimos a prueba el programa bajo tres situaciones comunes del mundo real.

4.4.1. Escenario 1: Venta normal con stock suficiente

Es el camino ideal del sistema. Empezamos con el producto 101 que tiene 50 unidades en stock.

- **Qué hace el sistema:** Java ejecuta la llamada para vender 5 unidades. El procedimiento guarda la venta y el disparador se activa al instante, restando 50 - 5.
- **Resultado:** Como hay suficiente stock, la base de datos guarda todo permanentemente. Al revisar las tablas, vemos que la venta quedó anotada y el stock bajó a 45 unidades sin problemas.

4.4.2. Escenario 2: Intento de venta sin stock suficiente

Esta prueba sirve para comprobar la seguridad del sistema cuando un usuario intenta comprar más unidades de las que existen. Recordando que pusimos la regla CHECK (stock >= 0) en la tabla.

- **Qué hace el sistema:** Intentamos registrar una venta de 60 unidades cuando solo nos quedan 50 en el almacén. El procedimiento guarda la venta temporalmente, pero cuando el disparador intenta restar 50 - 60 = -10, el motor de MySQL nota que esto rompe la regla de no tener stock negativo.
- **Resultado:** MySQL cancela inmediatamente toda la operación y borra la venta que intentaba guardarse (hace un *rollback*). Al mismo tiempo, le envía un mensaje de error a Java. La aplicación captura ese error en el bloque catch y le muestra al usuario el mensaje: *"Error crítico de persistencia: Violación de*

restricción de stock" de manera controlada y sin colgarse.

4.5. Síntesis Integradora del Capítulo

Poner a prueba estos tres escenarios demuestra por qué es una excelente decisión dejar que la base de datos controle las operaciones importantes. Mientras que una venta normal funciona de forma rápida, las situaciones de ventas seguidas o sin stock demuestran que MySQL actúa como un escudo que no permite que los datos se corrompan o queden con números negativos, haciendo que la aplicación en Java esté mucho más protegida y libre de errores lógicos complejos.

CAPÍTULO V

RESULTADOS

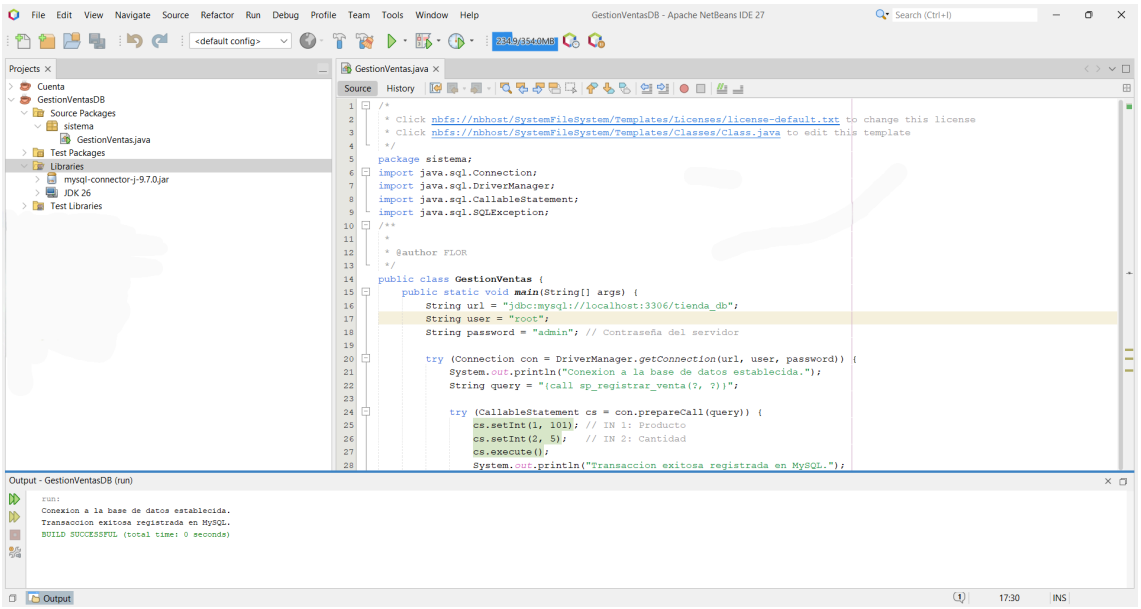
Tras la ejecución del caso de estudio implementado en el entorno MySQL–Java, se obtuvieron los siguientes hallazgos verificables:

5.1. Ejecución de la aplicación Java y conexión JDBC

- La aplicación Java compiló correctamente en el entorno JDK 25.
- Se estableció conexión con la base de datos MySQL mediante el driver mysql-connector-j.
- Se ejecutó la invocación del procedimiento almacenado sp_registrar_venta desde la interfaz CallableStatement.
- En la consola del sistema se obtuvo el mensaje de ejecución exitosa del

programa.

Figura 3: Ejecución de la aplicación Java en NetBeans



5.2. Registro de transacción en la base de datos

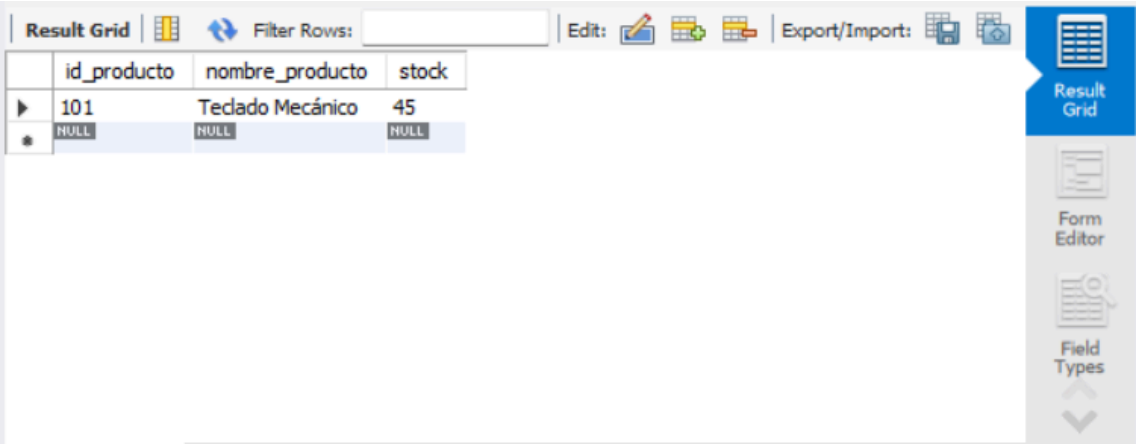
- Se registró una nueva fila en la tabla Ventas con los siguientes valores:
    - id_producto: 101
    - cantidad_vendida: 5
    - fecha de registro: automática (NOW())
- La operación fue confirmada mediante consulta SQL directa a la tabla Ventas:

SELECT * FROM Ventas;

5.3. Actualización del inventario en la tabla Productos

- Se verificó la modificación automática del campo **stock** en la tabla Productos.
- El valor inicial del producto (50 unidades) fue actualizado a 45 unidades después de la ejecución del sistema.

Figura 4: Verificación del stock actualizado en MySQL



5.4. Evidencia del comportamiento transaccional del sistema

- La ejecución del procedimiento almacenado generó una inserción en la tabla Ventas.
- El disparador trg_actualizar_stock se ejecutó automáticamente tras la inserción.
- Se observó consistencia entre el registro de venta y la actualización del inventario en una sola ejecución del sistema.

CAPÍTULO VI

DISCUSIÓN

Los resultados obtenidos en el sistema de persistencia transaccional permiten realizar una interpretación crítica en relación con la literatura revisada, especialmente en lo referente al uso de procedimientos almacenados, disparadores y mecanismos de conexión JDBC.

En primer lugar, los hallazgos confirman lo planteado por Horstmann, Core Java Volume II, quien sostiene que el uso de interfaces como CallableStatement mejora la

seguridad en la ejecución de rutinas almacenadas al evitar la construcción dinámica de consultas SQL. En el mismo sentido, Oracle Java Tutorials enfatiza que la invocación de procedimientos mediante JDBC permite una separación clara entre la lógica de aplicación y la lógica de base de datos, reduciendo riesgos de inyección SQL y errores de sintaxis en tiempo de ejecución.

Por otro lado, desde la perspectiva de la base de datos, MySQL Documentation establece que los disparadores (triggers) permiten automatizar reglas de negocio directamente en el motor, garantizando consistencia inmediata en operaciones transaccionales. Esto se evidencia en el sistema implementado, donde la actualización del stock se ejecuta automáticamente tras cada inserción en la tabla Ventas, sin intervención del cliente.

Asimismo, Peña et al. Proceso de réplica de datos para Reko señala que la delegación de lógica al gestor de base de datos mejora la consistencia y reduce errores derivados de múltiples accesos concurrentes. Este comportamiento se valida en el caso de estudio, donde el stock se actualiza de forma atómica desde el motor InnoDB.

Sin embargo, otros autores advierten limitaciones importantes. Deitel & Deitel Cómo programar en Java y Bruce Eckel Piensa en Java coinciden en que el uso excesivo de lógica en la base de datos puede generar un acoplamiento fuerte con el motor SQL, reduciendo la portabilidad del sistema. Esta limitación se observa en el sistema desarrollado, ya que la lógica del negocio depende directamente de MySQL y requeriría reescritura si se migrara a otro SGBD.

En cuanto al rendimiento y organización del sistema, López y García Análisis de rendimiento en bases de datos relacionales sostiene que el uso de procedimientos almacenados puede mejorar el desempeño al reducir viajes de red entre cliente y

servidor. Este comportamiento es coherente con los resultados obtenidos, donde la ejecución del procedimiento y el disparador ocurre completamente dentro del motor de base de datos, minimizando la latencia.

Limitaciones del sistema

Durante la implementación se identificaron algunas restricciones técnicas:

- Dependencia del motor MySQL para la ejecución de lógica transaccional.
- Dificultad de escalabilidad si se migran reglas complejas al servidor.
- Falta de capa intermedia (como ORM) que podría mejorar la abstracción del modelo de datos.

Estas limitaciones coinciden con lo señalado por Horstmann, Core Java Volume II, quien advierte que el diseño de sistemas debe equilibrar eficiencia y mantenibilidad para evitar dependencias excesivas del SGBD.

Áreas de investigación futura

A partir de los resultados obtenidos, se plantean las siguientes líneas de investigación:

- Comparación entre acceso directo JDBC y frameworks ORM como Hibernate.
- Evaluación del rendimiento de triggers en entornos distribuidos o en la nube.
- Análisis del impacto de la lógica embebida en el motor de base de datos frente a arquitecturas microservicio.

CAPÍTULO VII

CONCLUSIONES Y RECOMENDACIONES

Conclusiones

1. Se analizaron exhaustivamente los objetos relacionales del servidor. Se comprobó que los procedimientos almacenados optimizan el flujo de red al reducir las peticiones del cliente, mientras que los disparadores actúan como vigilantes que protegen la integridad matemática de los registros críticos.
2. Se demostró empíricamente la creación de estos elementos en MySQL mediante delimitadores ($$) y variables de estado contextuales (NEW), solucionando efectivamente la latencia en transacciones comerciales complejas.
3. Se concluye con absoluta certeza que la instanciación de la interfaz CallableStatement del paquete java.sql es la metodología más efectiva y segura para comunicar ecosistemas Java con rutinas almacenadas en MySQL.

Recomendaciones

- Se recomienda a los equipos de desarrollo priorizar siempre el uso de CallableStatement en sistemas empresariales para aislar el núcleo de la base de datos de posibles vulnerabilidades originadas en la capa de presentación.
- Se sugiere implementar bloques try-catch con recursos (Try-with-resources) para la clase SQLException, garantizando el cierre automático de la conexión a la base de datos para evitar fugas de memoria (memory leaks).
- Se aconseja utilizar los disparadores con suma cautela y mantener un Diccionario de Datos rigurosamente actualizado, ya que su autonomía silenciosa complica enormemente la trazabilidad de errores durante las pruebas de software.

Referencias

Baeldung. (2023). *JDBC CallableStatement*. https://www.baeldung.com/

Deitel, P. J., & Deitel, H. M. (2012). *Cómo programar en Java*. Pearson

Educación.
https://www.mfbarcell.es/docencia_uned/fund_inf_ing/libros/-%20Como%20Programar%20en%20Java%20Deitel.pdf

Eckel, B. (2002). *Piensa en Java*. Prentice Hall.
https://www.mfbarcell.es/docencia_uned/poo/material_complementario/piensaenjava/Piensa%20en%20Java%20-%20Bruce%20Eckel.pdf

Giraldo Mejía, J. C., & Fernández Ledesma, J. D. (2011). *Metodología para la construcción de un migrador universal de bases de datos*. DYNA: Revista de la Facultad de Minas, 78(165), 266–275.
https://dialnet.unirioja.es/servlet/articulo?codigo=7761311

Horstmann, C. S. (2022). *Core Java, Volume II: Advanced Features*. Pearson Education.
https://ptgmedia.pearsoncmg.com/images/9780137871070/samplepages/9780137871070_Sample.pdf

López, M., & García, P. (2023). *Análisis de rendimiento en bases de datos relacionales usando rutinas almacenadas*. Revista Ibérica de Sistemas y Tecnologías de Información, (E55), 145–158. https://www.risti.xyz/

MySQLTutorial. (s. f.). *MySQL stored procedures and triggers*.
https://www.mysqltutorial.org/

Oracle. (2023). *Java tutorials: Calling stored procedures*.
https://docs.oracle.com/javase/tutorial/jdbc/

Oracle. (2024). *¿Qué es MySQL?*
https://www.oracle.com/latam/mysql/what-is-mysql/

Oracle. (2026). *MySQL 8.0 Reference Manual*. https://dev.mysql.com/doc/refman/8.0/en/stored-objects.html

Peña, N. M., Martín, M. M., Labrada, R., & Leyva, G. R. (2016). *Proceso de réplica de datos con Microsoft SQL Server para el replicador de datos Reko*. Revista Cubana de Ciencias Informáticas, 10(3), 128–142. http://scielo.sld.cu/pdf/rcci/v10n3/rcci13316.pdf

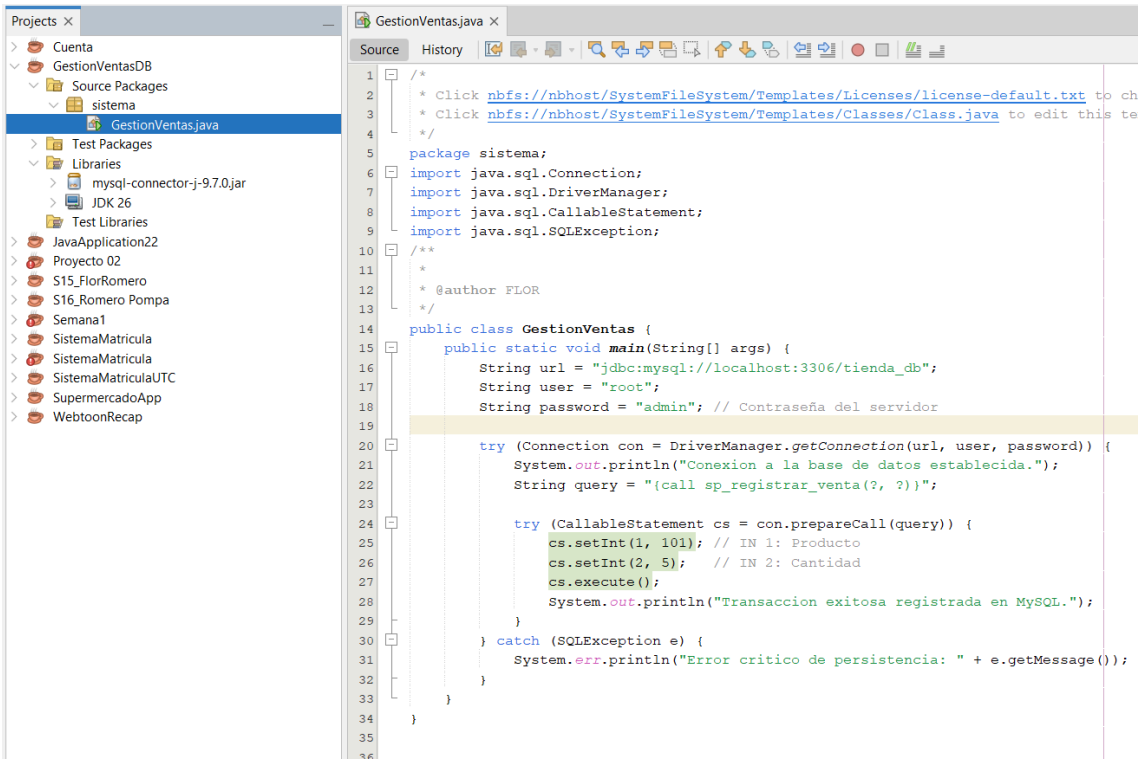
Quintero, J. B., Hernández, D. M., & Yanza, A. (2008). *Directrices para la construcción de artefactos de persistencia en el proceso de desarrollo de software*. Revista EIA, (9), 77–90. http://www.scielo.org.co/pdf/eia/n9/n9a07.pdf

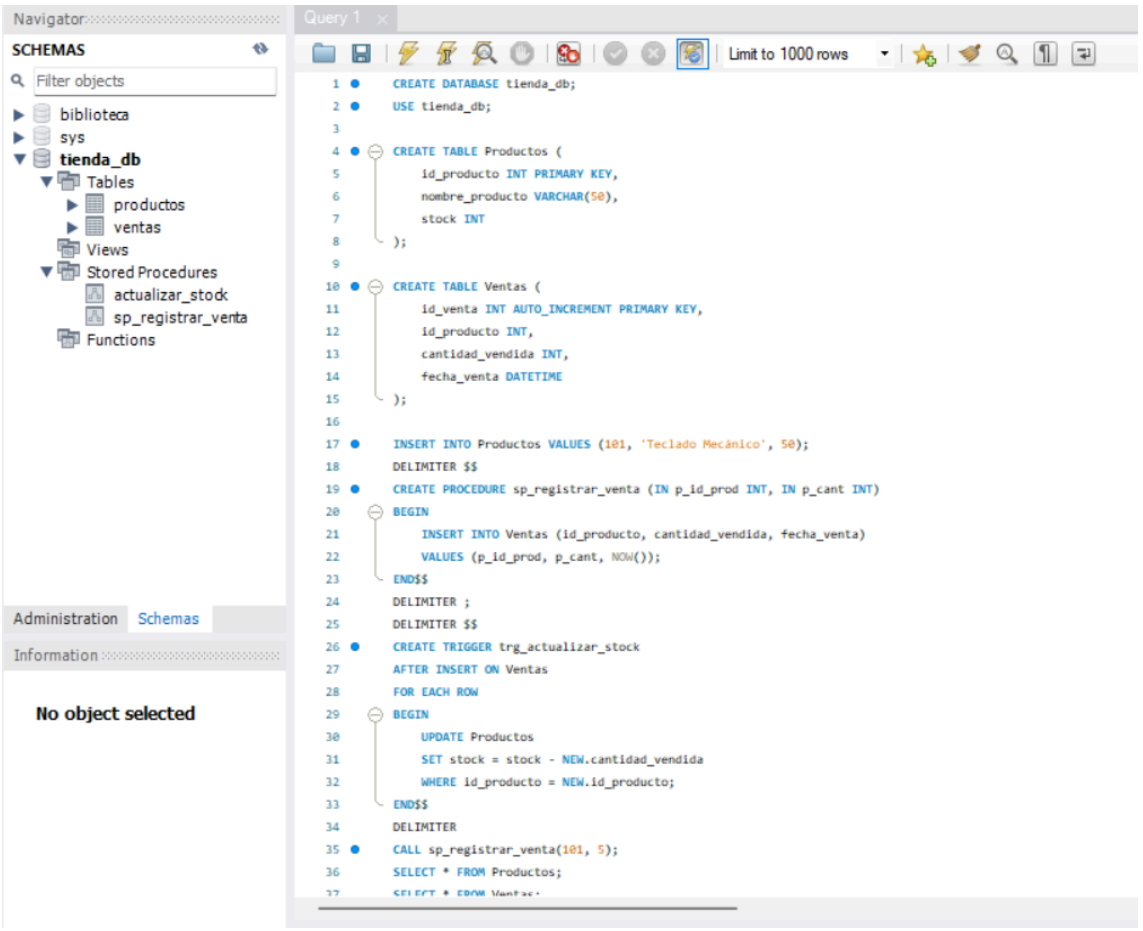
Apéndices

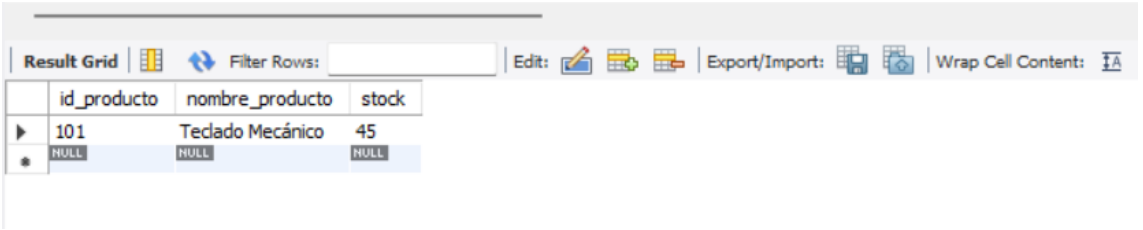
Apéndice A: Arquitectura Conceptual y Evidencias

Figura 4

Esquema conceptual de integración cliente-servidor







Nota. Modelo de capas utilizado para la interconexión entre Apache NetBeans y el motor MySQL Server.